

Repaso MA2015

Diseño de algoritmos matemáticos bioinspirados (Gpo 201)

Algoritmos Bioinspirados

Algoritmo Genético

Los algoritmos genéticos (AG) son una técnica de optimización inspirada en los principios de la evolución natural y la genética. Se basan en una población de posibles soluciones (individuos) que evolucionan a través de operadores biológicos simulados como la selección, el cruce y la mutación. Su objetivo es encontrar soluciones cercanas al óptimo global en problemas donde los métodos deterministas no son eficientes o factibles.

1. Inicialización de la población

¿Qué hace? Crea un conjunto inicial de soluciones candidatas que representan posibles respuestas al problema.

¿Cómo se hace? Se generan aleatoriamente los individuos de la población, donde cada uno codifica una solución en forma de cadena (genotipo), comúnmente binaria, entera o real.

¿Qué opciones existen?

- Generación completamente aleatoria.
- Inicialización mediante heurísticas para mejorar la calidad inicial.
- Mezcla de soluciones aleatorias y guiadas.

2. Evaluación de la población

¿Qué hace? Mide la calidad de cada individuo en función del problema a resolver.

¿Cómo se hace? Se utiliza una *función de aptitud* (fitness function) que calcula el valor objetivo asociado a cada solución.

¿Qué opciones existen?

- Maximizar la función de desempeño.
- Minimizar el error o costo.
- Normalizar valores para evitar dominancia de soluciones extremas.

3. Selección

¿Qué hace? Elige los individuos más aptos para reproducirse y generar la nueva población.

¿Cómo se hace? Se aplican métodos probabilísticos que otorgan mayor probabilidad de selección a los individuos con mejor desempeño.

¿Qué opciones existen?

- Ruleta (proporcional al fitness).
- Torneo (comparación entre subconjuntos).
- Rango o selección elitista.

4. Cruce o recombinación

¿Qué hace? Combina información genética de dos o más padres para crear nuevos individuos (descendientes).

¿Cómo se hace? Se elige un punto o varios puntos de corte en las cadenas de los padres y se intercambian segmentos para formar los hijos.

¿Qué opciones existen?

- Cruce de un punto.
- Cruce de dos puntos.
- Cruce uniforme.
- Cruce aritmético (para representaciones reales).

5. Mutación

¿Qué hace? Introduce pequeñas alteraciones en los individuos para mantener la diversidad genética.

¿Cómo se hace? Se modifica aleatoriamente uno o varios genes del cromosoma con una baja probabilidad.

¿Qué opciones existen?

- Inversión de bits (para codificación binaria).
- Perturbación aditiva (para valores reales).
- Permutación (para problemas combinatorios).

6. Reemplazo o actualización de la población

¿Qué hace? Forma la nueva población que participará en la siguiente generación.

¿Cómo se hace? Se decide cuáles individuos (padres e hijos) permanecerán para continuar el proceso evolutivo.

¿Qué opciones existen?

- Reemplazo generacional total.
- Reemplazo parcial con elitismo (manteniendo los mejores).
- Estrategias estacionarias (reemplazar pocos individuos por iteración).

7. Criterio de parada

¿Qué hace? Determina cuándo detener el proceso evolutivo.

¿Cómo se hace? Se evalúan condiciones de convergencia o límites de iteración.

¿Qué opciones existen?

- Número máximo de generaciones alcanzado.
- Estancamiento del fitness (sin mejora significativa).
- Logro de una solución con calidad deseada.

Resultado final: El mejor individuo de la última generación se considera la solución óptima o cuasi-óptima al problema.

Colonia de Hormigas

La **Colonia de Hormigas Artificial** es una metaheurística inspirada en el comportamiento colectivo de las hormigas. Las soluciones se *construyen estocásticamente* guiadas por dos fuentes de información: (i) el rastro de *feromona* almacenado en los arcos y (ii) una *información heurística* que mide la conveniencia local (típicamente $\eta_{i,j} = 1/d_{i,j}$). La interacción colectiva y la actualización iterativa de feromonas permiten que la colonia converja hacia rutas cada vez mejores.

1) Definición del espacio y representación

¿Qué hace? Modela el problema como un conjunto de nodos y arcos por donde las hormigas pueden construir rutas.

¿Cómo se hace? Se define un grafo con costos $d_{i,j}$ en los arcos; una solución es la ruta completa que recorre una hormiga tras n pasos.

¿Qué opciones existen? Nodos/ciudades y arcos/caminos con sus costos; elección de la métrica heurística local $\eta_{i,j}$ (por ejemplo $1/d_{i,j}$).

2) Inicialización

¿Qué hace? Prepara la colonia y el estado inicial de la memoria de feromonas.

¿Cómo se hace? Se dispone inicialmente de k hormigas y se fija un valor inicial para la feromona $\tau_{i,j} = \tau_0$ en todos los arcos (por ejemplo, $\tau_0 = 0.1$).

¿Qué opciones existen? Número de hormigas k ; valor inicial τ_0 ; parámetros del algoritmo como α, β, ρ, Q .

3) Regla de decisión local (movimiento probabilístico)

¿Qué hace? Determina a qué nodo moverse en cada paso durante la construcción de la ruta.

¿Cómo se hace? Desde el nodo i , la hormiga elige j con probabilidad

$$p_{i,j}^{(k)} = \frac{(\tau_{i,j})^\alpha (\eta_{i,j})^\beta}{\sum_{(i,\ell)} (\tau_{i,\ell})^\alpha (\eta_{i,\ell})^\beta},$$

donde $\tau_{i,j}$ es la feromona en (i, j) , $\eta_{i,j}$ la conveniencia (típicamente $1/d_{i,j}$), α controla la influencia de la feromona y β la de la heurística. Se evita visitar nodos no permitidos.

¿Qué opciones existen? Ajuste de α y β para balancear feromona vs. heurística; definición del vecindario factible en cada paso.

4) Construcción de soluciones

- ¿Qué hace?** Genera rutas completas para todas las hormigas en una iteración.
- ¿Cómo se hace?** A cada paso, cada hormiga elige una ciudad no visitada según la regla probabilística anterior; tras n pasos todas han completado su ruta.
- ¿Qué opciones existen?** Política para tratar restricciones (construcción sólo factible o permitir no factibles si es necesario).

5) Evaluación de soluciones

- ¿Qué hace?** Mide la calidad de cada ruta construida.
- ¿Cómo se hace?** Se calcula el costo L_k de la ruta de la hormiga k (suma de $d_{i,j}$ de los arcos recorridos).
- ¿Qué opciones existen?** Elección de la métrica de costo/beneficio que alimentará el depósito de feromona.

6) Actualización de feromonas (evaporación + depósito)

- ¿Qué hace?** Ajusta la memoria colectiva para intensificar buenos caminos y atenuar el resto.
- ¿Cómo se hace?** Una vez que todas las hormigas han completado sus rutas,

$$\tau_{i,j} \leftarrow (1 - \rho) \tau_{i,j} + \sum_k \Delta\tau_{i,j}^{(k)}, \quad \Delta\tau_{i,j}^{(k)} = \begin{cases} \frac{Q}{L_k}, & \text{si la hormiga } k \text{ usó } (i, j), \\ 0, & \text{en otro caso.} \end{cases}$$

- Aquí ρ es el coeficiente de evaporación, Q una constante y L_k el costo de la ruta de la hormiga k .
- ¿Qué opciones existen?** Elección de ρ (tasa de evaporación) y Q (intensidad del refuerzo); depósito acumulado por todas las hormigas según sus recorridos.

7) Acciones independientes (centralizadas)

- ¿Qué hace?** Implementa acciones globales que una sola hormiga no puede realizar.
- ¿Cómo se hace?** Se aplican procedimientos centralizados después de la construcción/actualización para influir la búsqueda de la colonia.
- ¿Qué opciones existen?** Diferentes acciones centralizadas según la estrategia elegida (definidas por el diseñador del algoritmo).

8) Iteración y criterio general

- ¿Qué hace?** Repite el ciclo de construcción–evaluación–actualización para mejorar progresivamente las rutas.
- ¿Cómo se hace?** Con las nuevas feromonas, se vuelve a iniciar la construcción de rutas por todas las hormigas y se repite el proceso.
- ¿Qué opciones existen?** Número de iteraciones o condición de paro basada en estabilidad/mejora de costos.

Colonia de Abejas

La **Colonia de Abejas Artificial (CAA / ABC)** es un algoritmo de optimización *estocástico* inspirado en la cooperación inteligente de las abejas durante la búsqueda de alimento. En ABC,

cada *fente de alimento* representa una solución candidata y la *cantidad de néctar* corresponde a su calidad (aptitud). Existen tres tipos de abejas: *obreras* (exploran vecindarios de sus fuentes), *observadoras* (seleccionan fuentes en función del fitness) y *exploradoras* (reemplazan fuentes estancadas con nuevas soluciones aleatorias).

1) Representación y evaluación (aptitud)

¿Qué hace? Define cómo se interpreta una fuente como solución y cómo se mide su calidad.

¿Cómo se hace? Se asocia cada fuente de alimento con una solución; su néctar es la calidad. Para minimización (p.ej., $f(x) = x^2$), puede usarse una transformación a fitness positivo, como $fit = \frac{1}{1+f(x)}$.

¿Qué opciones existen?

- Definir la función objetivo $f(\cdot)$ según el problema.
- Elegir una función de aptitud proporcional a calidad/néctar; para minimización, usar transformaciones positivas (p.ej., $1/(1+f)$).

2) Inicialización

¿Qué hace? Crea SN soluciones iniciales y fija parámetros del algoritmo.

¿Cómo se hace? Muestreo uniforme por dimensión:

$$x_{i,j} = x_j^{\min} + \text{rand}(0,1) (x_j^{\max} - x_j^{\min}), \quad i = 1, \dots, SN, \quad j = 1, \dots, D,$$

y se evalúa la función objetivo en cada x_i . Se establecen SN , D , **límite** (estancamiento) y # iteraciones.

¿Qué opciones existen?

- Tamaño de colonia SN , dimensión D , **límite** de no-mejora, número de iteraciones.
- Rangos $[x_j^{\min}, x_j^{\max}]$ para cada variable.

3) Fase de abejas *obreras* (búsqueda de vecindario)

¿Qué hace? Explora un vecino de cada fuente actual para intentar mejorarla.

¿Cómo se hace? Para cada x_i se elige al azar $k \neq i$ y (por dimensión j) se genera

$$v_{i,j} = x_{i,j} + \Phi (x_{i,j} - x_{k,j}), \quad \Phi \in [-1, 1],$$

se evalúa v_i y, si mejora, reemplaza a x_i (*greedy*).

¿Qué opciones existen?

- Elección aleatoria de $k \neq i$.
- Amplitud del paso controlada por $\Phi \in [-1, 1]$.
- Política de reemplazo codiciosa (sólo si mejora).

4) Fase de abejas *observadoras* (selección probabilística)

¿Qué hace? Asigna exploración adicional a fuentes prometedoras según su aptitud.

¿Cómo se hace? Se calculan probabilidades proporcionales al fitness:

$$p_i = \frac{f_i}{\sum_{\ell=1}^{SN} f_\ell},$$

cada observadora elige una fuente x_i con probabilidad p_i y aplica el mismo operador vecinal $v_{i,j} = x_{i,j} + \Phi(x_{i,j} - x_{k,j})$ con $k \neq i$, usando reemplazo si hay mejora.

¿Qué opciones existen?

- Esquema de selección proporcional p_i según aptitud.
- Reutilizar el operador vecinal de obreras y la misma regla de reemplazo.

5) Fase de abejas *exploradoras* (reemplazo por estancamiento)

¿Qué hace? Reintroduce diversidad cuando una fuente deja de mejorar.

¿Cómo se hace? Si una fuente no mejora durante un número fijo de intentos (**límite**), se *abandona* y la abeja asociada genera una nueva solución aleatoria en el dominio (misma fórmula de inicialización).

¿Qué opciones existen?

- Valor del **límite** (contador de no-mejora).
- Reposición aleatoria dentro de $[x^{\text{mín}}, x^{\text{máx}}]$.

6) Memorización de la mejor solución

¿Qué hace? Conserva la mejor solución encontrada hasta el momento para reporte y guía.

¿Cómo se hace? Al final de cada ciclo (obreras + observadoras + exploradoras) se actualiza el mejor global (*best-so-far*).

¿Qué opciones existen?

- Registrar y actualizar el mejor valor y su posición en cada iteración.

7) Parámetros y criterio de paro

¿Qué hace? Controla el avance del algoritmo y decide cuándo detenerse.

¿Cómo se hace? Se ejecutan iteraciones fijadas (o hasta tiempo máximo), con parámetros: SN , D , **límite**, número de iteraciones.

¿Qué opciones existen?

- Máximo de iteraciones (o evaluaciones).
- Umbral de mejora/no-mejora con el contador **límite**.

Enjambre de Partículas

La **Optimización por Enjambre de Partículas (PSO)** es un método heurístico para encontrar mínimos o máximos globales inspirado en el movimiento colectivo de bandadas/escuelas. Cada partícula combina su *tendencia individual* (memoria propia) con la *información social* (mejor hallazgo del enjambre) mediante una actualización de velocidad y posición.

1) Definición de partícula y enjambre

¿Qué hace? Establece la estructura básica de búsqueda: un conjunto de partículas que exploran el espacio de soluciones.

¿Cómo se hace? Cada partícula i mantiene: posición x_i , velocidad v_i , valor de la función objetivo en x_i , y su mejor posición histórica $\hat{x}_i$ (personal best). El enjambre mantiene el mejor global g .

¿Qué opciones existen?

- Tamaño del enjambre n .
- Dimensión del problema (longitud de x_i y v_i).

2) Inicialización del enjambre

¿Qué hace? Crea el estado inicial para comenzar la búsqueda.

¿Cómo se hace? Se generan n partículas con posiciones aleatorias dentro de los límites del dominio; típicamente se inicia v_i en cero (o valores pequeños).

¿Qué opciones existen?

- Estrategia de muestreo aleatorio en el rango permitido.
- Velocidad inicial (cero o pequeña aleatoria).

3) Evaluación de partículas

¿Qué hace? Mide la calidad de las posiciones actuales.

¿Cómo se hace? Se calcula la función objetivo en x_i y se actualiza $\hat{x}_i$ si la nueva posición mejora su histórico; se actualiza g si aparece un mejor global. Es necesario saber si se trata de *minimización* o *maximización*.

¿Qué opciones existen?

- Criterio de comparación acorde al objetivo (min o max).

4) Actualización de velocidad (dinámica de movimiento)

¿Qué hace? Define el cambio de dirección e intensidad del movimiento de cada partícula.

¿Cómo se hace? Se usa:

$$v_i(t+1) = w v_i(t) + c_1 r_1 (\hat{x}_i(t) - x_i(t)) + c_2 r_2 (g(t) - x_i(t)),$$

donde w es el *coeficiente de inercia*, c_1 el *coeficiente cognitivo*, c_2 el *coeficiente social*, y r_1, r_2 son valores aleatorios en $[0, 1]$ (por componente o escalares).

¿Qué opciones existen?

- Rango típico: $c_1 \in [0, 2]$ (recomendado 2), $c_2 \in [0, 2]$ (recomendado 2).
- r_1, r_2 como *vectores* (más fluctuación) o *escalares*.
- Balance exploración/explotación variando la relación c_1 vs. c_2 .

5) Interpretación de componentes de la velocidad

¿Qué hace? Controla el equilibrio entre explorar y converger.

¿Cómo se hace?

- **Inercia** $w v_i(t)$: mantiene la tendencia de movimiento; valores mayores favorecen explorar, menores aceleran la convergencia.
- **Cognitiva** $c_1 r_1 (\hat{x}_i - x_i)$: atrae hacia el mejor propio (independencia/exploración individual).
- **Social** $c_2 r_2 (g - x_i)$: atrae hacia el mejor global (convergencia colectiva).

¿Qué opciones existen?

- Ajustar w (p.ej., valores recomendados alrededor de 0.8–1.2).
- Enfatizar c_1 (más exploración y autonomía) o c_2 (más explotación y convergencia).

6) Actualización de posición

¿Qué hace? Mueve la partícula a su nueva ubicación candidata.

¿Cómo se hace?

$$x_i(t+1) = x_i(t) + v_i(t+1).$$

¿Qué opciones existen?

- Ninguna adicional en la fórmula; se aplican después manejos de límites/velocidad.

7) Control de velocidad y límites del dominio

¿Qué hace? Evita velocidades excesivas y salidas del espacio de búsqueda.

¿Cómo se hace?

- **Acotado de velocidad**: $|v_{i,j}| \leq v_{\max,j}$; ejemplo: $v_{\max,j} = k (x_j^{\max} - x_j^{\min})/2$, con $k \in [0.1, 1]$.
- **Corrección por límites**: si $x_{i,j}$ excede $[x_j^{\min}, x_j^{\max}]$, forzar al límite correspondiente y reiniciar $v_{i,j} = 0$.

¿Qué opciones existen?

- Elección de k y de $v_{\max}$ por dimensión.
- Estrategia al violar límites: *clamp* y reinicio de velocidad.

8) Programación del coeficiente de inercia

¿Qué hace? Modula el paso de exploración temprana a convergencia tardía.

¿Cómo se hace? Reducción lineal de w con la iteración t :

$$w_t = \frac{(w_{\max} - w_{\min})}{t_{\max}} (t_{\max} - t) + w_{\min},$$

con valores recomendados $w_{\max} = 0.9$ y $w_{\min} = 0.4$.

¿Qué opciones existen?

- Parámetros $w_{\max}$, $w_{\min}$ y $t_{\max}$.

9) Ciclo iterativo y criterio de paro

¿Qué hace? Repite evaluar–mover hasta alcanzar una condición de terminación.
¿Cómo se hace? Tras actualizar v y x , se reevalúa, se actualizan $\hat{x}_i$ y g , y se itera.
¿Qué opciones existen?

- Número máximo de iteraciones $t_{\text{máx}}$ o tiempo límite.
- Paro por convergencia (sin mejora del mejor global).

Modelos Matemáticos Comúnes

0.1. Partición de Grafo

Sea un grafo no dirigido $G = (V, E)$ con $|V| = N$ (par) y aristas E . Se desea particionar V en dos subconjuntos A y B de igual tamaño, minimizando las aristas que “cruzan” de A a B .

0.1.1.

section*Notación

- Variables binarias de asignación: $x_i \in \{0, 1\}$ para todo $i \in V$. $x_i = 1$ si el nodo i se ubica en la partición A , y $x_i = 0$ si se ubica en B .
- (Opcional) Pesos de arista $w_{ij} > 0$ para $(i, j) \in E$ (si el grafo es no ponderado, $w_{ij} = 1$).
- Variables de corte por arista: $y_{ij} \in [0, 1]$ indica si (i, j) queda cortada ($y_{ij} = 1$) o no ($y_{ij} = 0$).

Modelo de Programación Entera Mixta

$$\begin{aligned} \text{mín} \quad & \sum_{(i,j) \in E} w_{ij} y_{ij} \end{aligned} \tag{1}$$

$$\text{s.a.} \quad y_{ij} \geq x_i - x_j \quad \forall (i, j) \in E \tag{2}$$

$$y_{ij} \geq x_j - x_i \quad \forall (i, j) \in E \tag{3}$$

$$\sum_{i \in V} x_i = \frac{N}{2} \quad (\text{balance exacto}) \tag{4}$$

$$x_i \in \{0, 1\} \quad \forall i \in V \tag{5}$$

$$0 \leq y_{ij} \leq 1 \quad \forall (i, j) \in E. \tag{6}$$

Comentarios.

- Las restricciones (2)–(3) linealizan $y_{ij} = |x_i - x_j|$; al minimizar (22), y_{ij} toma 0 si ambos nodos quedan en la misma partición y 1 si quedan en distintas.
- La restricción (4) impone bisección perfecta; si se desea tolerancia δ , puede usarse $|\sum_i x_i - \frac{N}{2}| \leq \delta$, que se linealiza con dos desigualdades.
- (Antisimetría opcional) fijar un nodo, p.ej. $x_1 = 0$, para romper la simetría A/B .

Configuración breve de un Algoritmo Genético (ejemplo)

Componente	Elección (ejemplo para $N = 16$)
Codificación	Cromosoma binario $x \in \{0, 1\}^{16}$ con restricción $\sum_i x_i = 8$. Reparación post-cruce para reequilibrar si es necesario.
Población inicial	40 individuos, generados aleatoriamente y luego reparados para cumplir balance.
Función de aptitud	$\text{fitness}(x) = -\sum_{(i,j) \in E} w_{ij} x_i - x_j $ (maximizar) o equivalente de minimización del corte.
Selección	Torneo de tamaño 3 (con reemplazo).
Cruce	Binario de 1-2 puntos ($p_c = 0.9$) + reparación para restablecer $\sum_i x_i = 8$.
Mutación	Volteo de bit con tasa $p_m = \frac{1}{N}$ y posterior reparación de balance si se viola.
Elitismo	Conservar los mejores 2 individuos por generación.
Criterio de paro	200 generaciones o 30 generaciones sin mejora del mejor.

Cuadro 1: Configuración GA concisa para el problema de bisección de grafo.

TSP

Sea un conjunto de ciudades $V = \{1, \dots, n\}$ con $n = 7$ y costos (distancias) $c_{ij} \geq 0$ para viajar de i a j ($i \neq j$). Se busca un tour mínimo que inicie en la ciudad 1, visite cada ciudad exactamente una vez y regrese a 1.

Variables

- $x_{ij} \in \{0, 1\}$ para $i \neq j$: vale 1 si se viaja directamente de i a j .
- $u_i \in [2, n]$ para $i = 2, \dots, n$: posición de visita (variables MTZ) para eliminar subciclos.

Modelo (MTZ, dirigido)

$$\text{mín} \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n c_{ij} x_{ij} \quad (7)$$

$$\text{s.a.} \sum_{\substack{j=1 \\ j \neq i}}^n x_{ij} = 1 \quad \forall i \in V \quad (\text{sale una vez}) \quad (8)$$

$$\sum_{\substack{j=1 \\ j \neq i}}^n x_{ji} = 1 \quad \forall i \in V \quad (\text{entra una vez}) \quad (9)$$

$$u_i - u_j + n x_{ij} \leq n - 1 \quad \forall i \neq j, \quad i \geq 2, \quad j \geq 2 \quad (\text{MTZ}) \quad (10)$$

$$u_1 = 1, \quad 2 \leq u_i \leq n \quad \forall i = 2, \dots, n \quad (11)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \neq j. \quad (12)$$

Notas.

- Las ecuaciones (8)–(9) fuerzan un ciclo hamiltoniano; (10)–(11) eliminan subciclos.
- Si el grafo no es completo, fije $x_{ij} = 0$ cuando la arista $i \rightarrow j$ no exista.
- Para TSP simétrico puede usarse una versión no dirigida con $x_{ij} = x_{ji}$ y grados 2 por nodo.

Genético propuesto

Componente	Elección (ejemplo)
Codificación	Permutación de las ciudades $[1, \pi_2, \dots, \pi_7]$; el tour se cierra regresando a 1. Se fija la primera posición en 1 y se permutan sólo las restantes.
Población inicial	50 permutaciones factibles (aleatorias) respetando inicio fijo en 1.
Función de aptitud	$\text{fitness}(\pi) = -\sum_{k=1}^n c_{\pi_k, \pi_{k+1}}$ con $\pi_{n+1} = 1$ (equivale a minimizar la longitud).
Selección	Torneo de tamaño 3 o ruleta por ranking.
Cruce	Operadores para permutaciones: OX (Order Crossover) o PMX; prob. $p_c = 0.9$.
Mutación	<i>Swap</i> , <i>inversion</i> (2-opt) o <i>insert</i> ; prob. $p_m = 1/(n-1)$.
Reparación	Asegurar inicio en 1 tras cruce/mutación; eliminar duplicados y reinsertar faltantes.
Elitismo	Conservar los mejores $e = 2$ individuos por generación.
Criterio de paro	300 generaciones o 40 generaciones sin mejora del mejor.

Cuadro 2: GA conciso para TSP con 7 ciudades (origen fijo en ciudad 1).

0.2. Secuenciación de Tareas

Modelación y Solución Óptima

Parámetros:

- n : número de tareas,
- t_j : fecha de entrega de la tarea j ,
- h_j : costo unitario por retención de la tarea j ,
- c_j : costo unitario por penalización de la tarea j ,
- M : constante muy grande.

Variables de decisión:

- $S_j \geq 0$ tiempo de inicio de la tarea j ,
- $F_j \geq 0$ tiempo de finalización de la tarea j ,
- $E_j \geq 0$ tiempo de retención de la tarea j ,
- $T_j \geq 0$ tiempo de penalización de la tarea j ,
- $x_{ij} \in \{0, 1\}$ 1 si la tarea i se programa antes que j .

Función objetivo:

$$\min \sum_{j=1}^n (h_j E_j + c_j T_j) \quad (13)$$

Sujeto a:

1. *Definición del tiempo de finalización:*

$$F_j = S_j + t_j \quad \forall j = 1, \dots, n \quad (14)$$

2. *Definición de tiempos de retención y penalización:*

$$E_j \geq d_j - F_j \quad \forall j = 1, \dots, n \quad (15)$$

$$T_j \geq F_j - d_j \quad \forall j = 1, \dots, n \quad (16)$$

$$E_j, T_j \geq 0 \quad \forall j = 1, \dots, n \quad (17)$$

3. *Restricciones de no solapamiento:*

$$S_i + p_i \leq S_j + M(1 - x_{ij}) \quad \forall i \neq j \quad (18)$$

$$S_j + t_j \leq S_i + Mx_{ij} \quad \forall i \neq j \quad (19)$$

$$x_{ij} + x_{ji} = 1 \quad \forall i \neq j \quad (20)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \neq j \quad (21)$$

Algoritmo Genético

Tipo de cromosoma:	Orden de ejecución de tareas
Longitud:	5 genes
Criterio de inicialización:	Conjunto de permutaciones aleatorias
Criterio de infactibilidad:	No se puede repetir una misma tarea en la secuencia
Criterio de Paro:	Máximo 200 generaciones o 30 generaciones sin mejora
Función fitness:	Función objetivo 13 evaluado recorriendo la secuencia
Criterio de selección:	Torneo binario
Tamaño de la población:	30 individuos
Probabilidad de cruce:	0.9
Puntos de cruce:	Dos puntos
Lugar de cruce:	Intermedio preservando orden relativo
Probabilidad de mutación:	0.1
Criterio de reemplazo:	Generacional con elitismo (el mejor sobrevive)

Cuadro 3: Configuración del algoritmo genético.

Coloración de Grafos

Modelación matemática (Graph Coloring)

Dado un grafo no dirigido $G = (V, E)$, se desea colorear cada nodo con el menor número de colores N_c tal que nodos adyacentes no compartan color.

*Variables Sea K una cota superior al número de colores ($K \leq |V|$).

- $x_{vk} \in \{0, 1\}$ para $v \in V$, $k = 1, \dots, K$. Vale 1 si el nodo v usa el color k .

- $y_k \in \{0, 1\}$ para $k = 1, \dots, K$. Vale 1 si el color k es utilizado por al menos un nodo.

*Modelo de Programación Entera Binaria

$$\text{mín} \quad \sum_{k=1}^K y_k \quad (22)$$

$$\text{s.a.} \quad \sum_{k=1}^K x_{vk} = 1 \quad \forall v \in V \quad (\text{cada nodo recibe un color}) \quad (23)$$

$$x_{ik} + x_{jk} \leq y_k \quad \forall (i, j) \in E, \forall k \quad (\text{adyacentes no comparten color}) \quad (24)$$

$$x_{vk} \leq y_k \quad \forall v \in V, \forall k \quad (\text{activar color si se usa}) \quad (25)$$

$$y_k \geq y_{k+1} \quad \forall k = 1, \dots, K-1 \quad (\text{romper simetrías}) \quad (26)$$

$$x_{vk} \in \{0, 1\}, y_k \in \{0, 1\} \quad (27)$$

Notas.

- La restricción (24) prohíbe que dos nodos adyacentes tomen el mismo color (si $y_k = 1$). Si se prefiere, puede usarse la forma más fuerte $x_{ik} + x_{jk} \leq 1$ y (25) solo para enlazar con y_k .
- (26) reduce la simetría imponiendo que si se usa el color $k + 1$, entonces también se usa el k .
- Si K es suficientemente grande, el óptimo de (22) entrega N_c .

Configuración breve de un Algoritmo Genético (ejemplo)

Componente	Elección (ejemplo)
Codificación	Vector entero $c = (c_1, \dots, c_{ V })$ con $c_v \in \{1, \dots, K\}$.
Población inicial	80 individuos: asignaciones aleatorias seguidas de un <i>greedy</i> local para reducir conflictos.
Función de aptitud	Minimizar penalizada: $\text{cost}(c) = \# \text{conflictos}(c) + \lambda \# \text{colores_usados}(c)$. Fitness = $-\text{cost}$. $\lambda > 0$ controla reducción de colores.
Selección	Torneo de tamaño 3 (con reemplazo) o ruleta por ranking.
Cruce	Uniforme por genes o PMX por bloques de nodos; prob. $p_c = 0.9$.
Mutación	Recoloreo de un nodo conflictivo (cambio a color factible/menos conflictivo) o intercambio de colores; prob. $p_m = 1/ V $.
Reparación	Heurística de recoloreo local (Kempe/greedy) si quedan conflictos tras cruce/mutación.
Elitismo	Conservar los mejores $e = 2$ individuos por generación.
Control de K	Estrategia <i>schedule</i> : iniciar con K_0 alto y reducir K una vez que se logre solución factible persistente.
Criterio de paro	500 generaciones o 50 sin mejora; o alcanzar 0 conflictos con mínimo K objetivo.

Cuadro 4: Configuración GA concisa para el problema de coloreado de grafos.